HASHING TECHNIQUE

1. Hashing?

Hashing is a technique used to store and search data quickly by converting a key into an index (address) of an array using a hash function.

In simple words, hashing helps us find data quickly without checking every element one by one.

Real-life example

Suppose a college has 1,000 students. Each student has a unique roll number.

Instead of checking every student's record, we use a calculation to find the location where a particular student's record is stored. This process is called hashing.

HOW HASHING WORKS

Key = 123

Hash Function

h(k) = (k % 10), then we get Index = 3

Here, 123 mod = 3, because the remainder is 3. It means Data is stored at index 3.

Therefore, the data associated with key 123 is stored at index 3.

2. Hash Table?

A hash table is a data structure that stores data in an array-like structure and uses a hash function to calculate where each item should be stored.

A hash table generally stores data in the form of key-value pairs.

For example:

Key (Roll No.)	**Value (Student Name)**
101	Amit
102	Ravi
103	Neha

Here, the roll number is the key, and the student's name is the value.

Example of a hash table

Suppose the hash table has 10 locations, from index 0 to index 9.

We use this hash function: h(k)=k % 10

Let us insert three keys: 21, 32, 45 and 14.

- For key 21: (21% 10 =1)
- For key 32: (32% 10 =2)
- For key 45: (45% 10 =5)
- For key 32: (14% 10 =4)

The resulting hash table is:

Indexes 0, 3, 6, 7, 8 and 9 are also empty.

Note- The index is the position in the table, while the key is the actual value used to calculate that position.

Basic operations of a hash table-

1. Insertion: Add a new key-value pair.
2. Searching: Find a value using its key.
3. Deletion: Remove an existing key-value pair.
4. Updating: Change the value associated with a key.

The average time complexity of insertion, searching and deletion is generally (O(1), meaning constant time, when the hash function distributes keys well and collisions are handled efficiently. In the worst case, these operations can take (O(n).

3. Hash Function?

A hash function is a mathematical function that converts a key into an index(or a hash value) of a hash table.

It is commonly represented as: h(k) = Index value

Where:

- h = hash function
- k = key
- Index = position in the hash table

Example 1: Division method

Suppose the table size is 10.

The hash function is: h(k) = k % 10 (or k mod 10)

Here, mod means the remainder after division.

Calculate the indexes for the following keys: 15, 27, 34, 48, 52, 69

Key	Calculation	Index
15	(15 mod 10)	5
27	(27 mod 10)	7
34	(34 mod 10)	4
48	(48 mod 10)	8
52	(52 mod 10)	2
69	(69 mod 10)	9

For example, 27 divided by 10 gives a remainder of 7. Therefore, key 27 is mapped to index 7.

Example 2: Hash function with table size 7

h(k) = k % 7

Find the indexes of 10, 18, 25, 15 and 31.

Key	Calculation	Index
10	(10 % 7)	3
18	(18 % 7)	4
25	(25 % 7)	4
15	(15 % 7)	1
31	(31 % 7)	3

Notice something important:

- Keys **18 and 25** both map to index 4.
- Keys **10 and 31** both map to index 3.

Note- This situation is called a collision.

4. What is a Collision in Hashing?

A collision occurs when two or more different keys produce the same index through a hash function.

For example, using: h(k) = k mod 10 (or k % 10)

We get:

h(15) =5
h(45) =5

Both keys want to occupy index 5. This is a collision.

Collisions are normal in hashing and must be handled using a suitable technique.

5. Collision Handling Techniques

The two important methods are:

- Separate chaining or Chaining (called Open Hashing)
- Open addressing (called Closed Hashing)

Note- Open addressing includes linear probing, quadratic probing and double hashing.

A. Separate chaining

In separate chaining, each index maintains a list of keys that map to that index.

Example: Insert 15, 25 and 35 using h(k) = k % 10.

All three keys map to index 5.

Index 0 : Empty
Index 1 : Empty
Index 2 : Empty
Index 3 : Empty
Index 4 : Empty
Index 5 : 5 → 15 → 35 → NULL
Index 6 : Empty
Index 7 : Empty
Index 8 : Empty
Index 9 : Empty

All three keys are stored in a linked list associated with index 5.

Advantage: Multiple keys can be stored at the same index using a list.

B. Linear probing

In linear probing, if the calculated index is occupied, we check the next index until an empty location is found.

Formula: $hi(k) = (h(k) + i) \% m$

Where:

- h(k) = original hash index
- i = number of attempts, starting from 0 (also called prob value)
- m = table size

Example: Table size = 10 and h(k) = k % 10 .

Insert 15, 25 and 35.

Key	Original index	Final index
15	5	5
25	5	6
35	5	7

Now keys stored at :

1. 15 is stored at index 5.
2. 25 also gets index 5, but it is occupied. So, 25 is stored at index 6.
3. 35 gets index 5, but indexes 5 and 6 are occupied. So, 35 is stored at index 7.

Final hash table-

—	0
—	1
—	2
—	3
—	4
15	5
25	6
35	7
—	8
—	9

Disadvantage: Linear probing can create clusters of occupied positions, which may increase searching time.

C. Quadratic probing

In quadratic probing, we check positions using square increments instead of checking each next index.

A common formula is: h_i(k) = [h(k)+i^2) % m]

If the original index is 5, the sequence of possible positions is:

- First attempt: (5 + 0^2 = 5)
- Second attempt: (5 + 1^2 = 6)
- Third attempt: (5 + 2^2 = 9)
- Fourth attempt: (5 + 3^2 = 14), giving index 4 for a table of size 10.

Quadratic probing can reduce primary clustering, but its effectiveness depends on the table size and probing design.

D. Double hashing-

Double hashing uses two hash functions to calculate the next position when a collision occurs.

A common formula is: h_i(k) = [(h_1(k)+i ,h_2(k)) % m]

For example, use a table size of 7:

h_1(k) =[k % 7]
h_2(k) =[1 + (k % 5)]

For key 10:

h_1(10) =[3,quad h_2(10) =1]

The first position is 3. If index 3 is occupied, the next position is:

[(3+1) % 7 = 4]

If index 4 is also occupied, the next position is:

[(3+2) % 7 = 5]

Double hashing can distribute keys more effectively than simple linear probing when the hash functions are chosen well.

6. Properties of a Good Hash Function: A good hash function should have these properties:

1. Fast to calculate: It should calculate an index quickly.
2. Uniform distribution: It should distribute keys across the table as evenly as possible.
3. Fewer collisions: It should minimize the number of keys mapping to the same index.
4. Valid index: The calculated index must be within the range of the hash table.
5. Consistent result: The same key should produce the same hash value when the function and its relevant parameters remain unchanged.

7. Advantages and Disadvantages of Hashing

Advantages	**Disadvantages**
Very fast average searching	Collisions may occur
Fast insertion and deletion on average	Performance may degrade with many collisions
Useful for large collections of data	Data is not naturally stored in sorted order
Efficient key-value lookup	Resizing and rehashing may be required

8. Applications of hashing- Hash tables are commonly used in:

- Dictionaries and maps in programming languages.
- Database indexing and lookup structures.
- Compiler symbol tables.
- Caching systems.
- Password verification systems, using cryptographic hash functions.
- Counting the frequency of words or numbers.

Note:

- Hashing is the technique.
- Hash function is the calculation.
- Hash table is the storage structure.

9. Important solved example for examinations

Question: Insert the keys 12, 22, 32 and 42 into a hash table of size 10 using the division method and linear probing.

Given: h (k) = k % 10

Step 1: Calculate the hash indexes.

- h(12) = 2 (12 % 10)
- h(22) = 2 (22 % 10)
- h(32) = 2 (32 % 10)
- h(42) = 2 (42 % 10)

Step 2: Resolve collisions using linear probing.

- 12 is stored at index 2.
- 22 is stored at index 3 because index 2 is occupied.
- 32 is stored at index 4 because indexes 2 and 3 are occupied.
- 42 is stored at index 5 because indexes 2, 3 and 4 are occupied.

Final table:

Index	Stored key
0	Empty
1	Empty
2	12
3	22
4	32
5	42
6	Empty
7	Empty
8	Empty
9	Empty

This is a classic example of collision resolution using linear probing.

10. What is Load Factor in Hashing?

Definition: The load factor is the ratio of the number of elements stored in a hash table to the total number of available slots in that table.

It tells us how full a hash table is.

Formula- alpha = (n/m)

Where:

- alpha = Load factor
- n = Number of elements stored in the hash table
- m = Total number of slots in the hash table

Note- Remember that: Load factor = Number of stored elements ÷ Total number of slots.

Example 1. Suppose a hash table has 10 slots and we have stored 5 elements.

Alpha =5 /10 = 0.5

Therefore, the load factor = 0.5 (or 50%).

This means 50% of the table's slots are occupied.(here, Hash table with 10 slots)

15 Index 0

25 Index 1

35 Index 2

45 Index 3

55 Index 4

Empty Index 5

Empty Index 6

Empty Index 7

Empty Index 8

Empty Index 9

It means in hash table occupied positions: 5 of 10. (i.ei **50%)**

Example 2. Suppose a hash table has 20 slots and 15 elements.

Alpha = frac{15}/{20} = 0.75

The load factor is 0.75 (or 75%).

This means 75% of the slots are occupied and 25% are empty.

Example 3. Suppose a hash table has 10 slots and 12 elements.

Alpha = frac{12}/{10} = 1.2

The load factor is 1.2 (or 120%).

This is possible when the hash table uses separate chaining, because multiple elements can be stored at the same index in a linked list. In a basic open-addressing table, the load factor cannot exceed 1.

<u>Question</u> 1. Why is Load Factor Important?

1. Low load factor: More empty slots are available, and collisions are generally less frequent.
2. High load factor: The table is more crowded, so collisions or longer search sequences are more likely.
3. Rehashing: When the load factor becomes too high, a hash table may be resized and its elements inserted again using the new table size.

For example, some hash table implementations resize when their load factor exceeds a chosen threshold, such as 0.75. This is a design choice, not a universal rule.

Question 2. Define load factor and calculate it for a hash table containing 8 elements with a capacity of 16 slots.

Solution: alpha =frac{n}/{m} = frac{8}/{16} = 0.

It means, the load factor is 0.5 or 50%.

Remember that: Load factor = Number of stored elements ÷ Total number of slots.

Question 3. What is Hash Table?

" Hash Table is a data structure that stores data in an array-like structure and uses a hash function to calculate the location (index) where each element should be stored."

In simple words, a hash table helps us store, search, insert, and delete data quickly.

Real-life example- Suppose a college stores student's details using their roll numbers.

Roll Number (Key)	**Student Name (Value)**
101	Amit
102	Ravi
103	Neha
104	Priya

If we want to find the details of student 103, a hash function helps us locate the appropriate position directly instead of checking every student's record.

Example of a Hash Table- Suppose we have a hash table of size 10, with indexes from 0 to 9.

We use this hash function: h (k) = k mod 10 OR- h (k) = k % 10

Here, mod means the remainder after division.

Let us insert the keys 12, 25, 33, and 47.

Calculations:

- 12 % 10 = 2
- 25 % 10 = 5
- 33 % 10 = 3
- 47 % 10 = 7

Therefore, the hash table looks like this: (Hash Table Size = 10)

Empty Index 0
Empty Index 1

12	Index 2
33	Index 3
Empty	Index 4
25	Index 5
Empty	Index 6
47	Index 7
Empty	Index 8
Empty	Index 9

Explanation: Key 12 is stored at index 2, key 25 at index 5, key 33 at index 3, and key 47 at index 7.

Question 4. What happens if two keys have the same index?

Solution- This situation is called a collision. For example, using the hash function: h(k) = k %10

- 15 % 10 = 5 , and 25 % 10 = 5

Both keys want to occupy index 5. We need a collision-handling technique, such as linear probing or separate chaining, to store both keys correctly.

Question 5. Main operations of a Hash Table?

1. Insertion: Add a new element.
2. Searching: Find an element using its key.
3. Deletion: Remove an element.
4. Updating: Change an existing value.

The average time complexity of insertion, searching, and deletion is generally \(O(1)\), assuming a good hash function and effective collision handling.

Question 6. Advantages of a Hash Table?

- Fast searching of data.
- Fast insertion and deletion on average.
- Useful for storing key-value pairs.
- Used in dictionaries, databases, caches, and compiler symbol tables.

“A hash table is a data structure that stores data in an array-like structure using a hash function to calculate the index of each element.”

For example, if h(k) = k mod 10, then key 23 is stored at index 3 because (23 mod 10 = 3).

Hash tables provide average O(1) time for insertion, searching, and deletion under suitable conditions.

Question 7. What is a Hash function? With suitable example.

“A Hash Function is a mathematical function that converts a given key into an index (location) in a hash table where the data can be stored or searched.”

In other words we can say that, a hash function tells us where to store data in a hash table.

The general representation is: h(k) = {Index}

Where:

- h = Hash function
- k = Key (the data used to calculate the location)
- Index = Position in the hash table

1: Division Method-

Suppose we have a hash table of size 10, with indexes from 0 to 9.

Our hash function is: h(k) = k mod 10 [or h(k) = k % 10]

Here, mod means the remainder after division.

Let us calculate the index for different keys.

Key	Calculation	Hash Index
12	12 % 10	2
25	25 % 10	5
33	33 % 10	3
47	47 % 10	7
58	58 % 10	8

Example 2: If Hash Table of Size 7.

Suppose the hash function is : h(k) = k % 7

Calculate the indexes of keys 10, 18, 24 and 31.

Key	Calculation	Index
10	10 % 7	3
18	18 % 7	4
24	24 % 7	3
31	31 % 7	3

Important observation: Keys 10, 24 and 31 all produce index 3.

This is called a collision. It occurs when two or more different keys generate the same index.

We can handle collisions using techniques such as linear probing or separate chaining.

Question 8. Properties of a Good Hash Function?

A good hash function should:

1. Be fast to calculate.
2. Distribute keys uniformly across the table.
3. Minimize collisions.
4. Produce a valid index within the table's range.
5. Give the same result for the same key when the function and its parameters remain unchanged.

Question 9. Difference between Hash Function and Hash Table?

Hash Function	**Hash Table**
Calculates the index.	Stores the data.
It is a calculation or rule.	It is a data structure.
Example: h(k) = k mod 10	An array of 10 slots. A[10]

Note - Hash functions are used to make searching, insertion, and deletion efficient in hash tables.

(Dr. Rajeev Kumar)

Question Bank Regarding II-Midterm (DSA)

S. No.	Section-A / B / C
1.	Hashing? Advantages and Disadvantages of Hashing, Applications of hashing. Hash Table, Basic operations of a hash table Hash Function, Load Factor in Hashing?
2.	**Collision Handling Techniques:** **1-** Open Hashing (Chaining **)** 2- Closed Hashing **(a**- Linear probing b- Quadratic probing c- Double hashing)
3.	BST?-Definition, Operations, Traversal methods- Preorder, Inorder and Postorder AVL Tree, Red-Black Tree , and Splay Tree
5.	Sorting? Bubble,Selection,Radix, Merge, and Quick sort Heap Sort? Min-Heap and Max-Heap
6.	Graphs?- Definition, Types, Adjacency Matrix ,Adjacency List Traversal methods-BFS / DFS
7.	
8.	